Titanic Survival Classification Using a Decision Tree

Name: Jefraim Immaneul I
Reg No: 26BEC0493

First, we downloaded the dataset from Kaggle, namely the training set and the test set. The training set contained categories such as PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket and Fare, and the test set contained the same categories with two more: Cabin and Embarked. Using this, we were able to identify that the desired output would be in the form of a binary classification, where 1 denotes “survived” and 2 denotes “dead”.

Now I knew that I had to classify using these categories with the decision tree model. To begin with, I created a Kaggle notebook and imported the given Titanic dataset into it. I imported the following libraries: os, glob, pandas as pd, matplotlib.pyplot as plt, from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree, from sklearn.model_selection import train_test_split, cross_val_score, and from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay.

The os library is used to find the relevant files by using pattern recognition of the names of the files/file paths. The pandas library was used to clean the notebook and fill in some missing data, such as sex, for the datasheet provided. The matplotlib.pyplot library draws the plots: the confusion matrix, the feature-importance bars and the tree diagram. DecisionTreeClassifier learns to generate an output from the training given using the dataset. plot_tree draws the trained tree as a diagram so that you can see the rules it learned. export_text was used to write commands so that it would be easier for me to keep track of the progress. train_test_split splits the labelled data into 80% for training and 20% for testing, so that accuracy is measured on passengers the model has not seen. cross_val_score is just a backup accuracy test. accuracy_score is the percentage of passengers predicted correctly. classification_report is a scorecard that shows how well the model did on each group separately, “Died” and “Survived”. confusion_matrix counts the correct and wrong predictions and forms a 2x2 grid with the following pairs: (true, survived), (false, survived), (false, died), (true, died). Finally, ConfusionMatrixDisplay draws the grid and colours it.

After importing the libraries, I loaded the CSV files containing the datasets into my Kaggle notebook. The basic principle of a decision tree model is to compare two values and ask, “Is this value bigger or smaller than a certain reference value?” Computers can only compare numbers, not words, so every column is converted to a number (0 and 1, to be precise).

How did we convert the columns? For sex, we replaced the words with 0 and 1. For name, we kept the title part, which many people share, and turned it into yes/no (1/0) columns. For ticket, it is almost the same: we found whether people share the same ticket or not. For SibSp and Parch, we did not convert them, as they were numbers to begin with, and so are Age and Fare. After the vectorization part was over, we took 20% of the data from the test set and used it for the validation of the training.

Then the ML training process finally started. The max_depth of the decision tree model trained was 4. This limits the model to asking no more than 4 questions; without this, the questioning could go on forever. Asking many questions is certainly bad, as the model will keep asking questions until the last value of the dataset is read, which makes it hard to relate to a new passenger. Four was chosen out of the infinite set of numbers that could have been chosen because it gave the highest validation accuracy (81%, to be exact). The validation score dropped after 4. min_samples_leaf=5 makes sure that at least 5 passengers share a trait, so it does not add a separate trait for each new passenger added to the dataset, hence reducing the complexity of the computation. random_state=42 is just a random number chosen to break ties and fix the random choices we get, so that the authenticity of the model remains constant.

After the training process was complete, the validation was initiated and the accuracy was tested. The following results were obtained: the training accuracy was 0.8371, the validation accuracy was 0.8101, and the 5-fold CV accuracy was 0.8159 +/- 0.0294. In the classification report, the Died group had a precision of 0.84, a recall of 0.85, an f1-score of 0.85 and a support of 110, while the Survived group had a precision of 0.76, a recall of 0.74, an f1-score of 0.75 and a support of 69. The accuracy was 0.81 with a support of 179. The macro average was 0.80 for precision, recall and f1-score, and the weighted average was 0.81 for precision, recall and f1-score, both with a support of 179.

The trained model made a priority list to calculate the survival rate. The list contains these categories in ascending order of priority: Title_Mrs, Age, Parch, TicketHasPrefix, Title_Rare, Fare, FamilySize and Title_Mr, with Title_Mr leading the list.